

From Trees to Trellises: State-Aware Viterbi Inference (SAVI)

Scaling Test-Time Compute via Semantic-State Search

Fixed Research Positioning Memo — v6

Core thesis: SAVI converts test-time scaling from generating more textual variants into exploring more distinct semantic states. The key search geometry changes from a tree of surface histories to a trellis of executable equivalence classes.

1. The Core Problem: Semantic Aliasing

As the field pivots toward test-time compute scaling, a structural bottleneck has emerged. Autoregressive language models search over token histories, not over the underlying executable states those histories represent. Textually different paths branch and remain separate even when they are semantically identical. This creates a tree of surface forms when the underlying reasoning space often has a trellis structure.

Semantic aliasing is the phenomenon in which textually different reasoning paths collapse to the same algebraic expression, program state, SQL result, proof state, or simulator state. For example, $x=1; y=2$ and $y=2; x=1$ can represent the same memory state; algebraically, $a*(b+c)$ and $a*b+a*c$ are equivalent under expansion. Text-based search wastes beam capacity on these aliases.

Primary hypothesis: Textual diversity is not reasoning diversity. The first scientific question is not whether SAVI beats every baseline; it is whether standard test-time scaling spends a material fraction of compute on duplicate semantic states.

2. Formal Framework

Definition 1 — Semantic Equivalence

Let $H_t = (x_1, \dots, x_t)$ be a history of generated reasoning chunks, where each x_t is a logical unit such as one algebraic step, one line of code, or one proof move. Let S be a semantic state space and $\Phi: H \rightarrow S$ be a deterministic semantic map into canonical states.

$$H_A \sim H_B \text{ iff } \Phi(H_A) = \Phi(H_B)$$

Each equivalence class $[H]$ is a semantic state. The equivalence relation is only as sound as Φ . For symbolic algebra, Φ can be close to exact under specified assumptions. For code, Φ should be treated as **bounded observational equivalence**, not mathematical equivalence.

Definition 2 — Three Merge-Rate Metrics

Do not use one merge-rate number for everything. Three distinct metrics capture different aspects of aliasing and SAVI's mechanism:

Metric	Definition	Interpretation
--------	------------	----------------

BAR_t Baseline beam aliasing rate	$\text{BAR}_t = 1 - \text{unique } \Phi(H) \text{ among baseline text beam} / B_t $	Measures whether the problem exists in standard text-space search. High BAR_t means a baseline beam is wasting slots on semantic duplicates.
CMR_t SAVI candidate merge rate	$\text{CMR}_t = 1 - \text{unique valid semantic states before pruning} / \text{valid candidates before pruning} $	Measures whether SAVI is actively merging duplicate candidate states online. This is the mechanism metric.
RSD_t Retained semantic diversity	$\text{RSD}_t = \text{unique semantic states retained after pruning} / K$	Measures how much of the beam is filled with distinct semantic states after SAVI pruning.

Metric fix: BAR_t diagnoses redundancy in baseline beams. CMR_t diagnoses the SAVI mechanism. RSD_t reports how much semantic diversity remains after pruning.

Definition 3 — SAVI Viterbi Recurrence

Let $V_t(S_t)$ be the best accumulated score for reaching canonical state S_t at step t :

$V_t(S_t) = \max_{x_t, S_{t-1}} [V_{t-1}(S_{t-1}) + \log P_{\text{fwd}}(x_t \mid \text{prompt}(S_{t-1}), \text{representative_history})) + \lambda * \log P_{\text{bwd}}(G=1 \mid S_t, \text{problem})]$		
subject to: $S_t = \Phi(S_{t-1}, x_t) \neq \text{invalid}$		
Mode	λ	Meaning
Forward SAVI	0	State merging only. No learned value model. This is the cleanest first-paper contribution.
Bidirectional SAVI	> 0	State merging plus goal-conditioned value guidance. Should be an extension or ablation after Forward SAVI works.

Definition 4 — Goal-Conditioned Value Model

$$P_{\text{bwd}}(G=1 \mid S_t, \text{problem})$$

P_{bwd} is a learned value function. It estimates whether the current canonical state is likely to lead to a correct solution under the task objective. It is not a future-token predictor and it should **not** be presented as a Data Processing Inequality claim.

Wording fix: Use the simple value-function framing: P_{bwd} scores intermediate states relative to a known task goal. Avoid claiming that this proves a general information-theoretic directionality theorem inside this paper.

Definition 5 — Markov Approximation Error

True Viterbi requires that the future depends only on the current state. LLMs condition on text histories, so two histories that map to the same semantic state may still induce different next-action distributions. SAVI is

therefore **Viterbi-inspired**, not exact Viterbi.

$$\text{epsilon}(H_A, H_B) = D[P_fwd(. \mid H_A) \parallel P_fwd(. \mid H_B)], \text{ where } \Phi(H_A) = \Phi(H_B)$$

Exact KL over chunk distributions may be difficult to compute. A practical diagnostic: sample top-M next chunks from both histories, map them through Φ , and compare induced next-state distributions using JS divergence, top-state overlap, or agreement rate. If divergence is large, reconstruct the LLM prompt from the canonical state instead of continuing from an arbitrary preserved history.

Corrected Proposition — Tree vs Trellis Search

A full unpruned reasoning tree grows as $O(N^T)$. Ordinary beam search avoids this exponential explosion by retaining K text histories per step, evaluating approximately $O(K \cdot N \cdot T)$ candidates. SAVI does **not** claim a universal asymptotic improvement over pruned beam search. Its advantage is different: the K retained slots are unique semantic states rather than surface-distinct text histories.

Defensible claim: Under the same beam width K , a baseline text beam may contain only $K \cdot (1 - \text{BAR}_t)$ unique states. SAVI enforces a stronger invariant: retained states are unique after merging. When aliasing is nonzero, SAVI redirects beam capacity from duplicate text forms to additional semantic states.

3. The SAVI Algorithm

- **Generate candidate chunks:** from each active semantic state, sample N next logical chunks.
- **Execute / canonicalize:** apply Φ to map each chunk into a new semantic state.
- **Discard invalid transitions:** remove parse failures, execution errors, timeouts, or logically illegal transformations.
- **Merge equivalent states:** group candidates by canonical state before pruning.
- **Keep the best representative:** for each unique state, retain the highest-scoring text trajectory or a canonical prompt.
- **Prune to K states:** keep the top- K unique semantic states for the next step.
- **Return final trajectory:** choose the highest-scoring valid final state under the task objective.

Invariant: At every step, the retained beam contains at most one representative trajectory per canonical semantic state. The beam is a set of semantic states, not a multiset of text strings.

4. Scope: Executable Domains

The first study should restrict scope to domains where state equivalence can be computed or approximated under explicit constraints.

Domain	Semantic map	Equivalence claim	First target
--------	--------------	-------------------	--------------

A. Symbolic algebra	SymPy canonicalization under declared assumptions.	Near-exact symbolic equivalence for restricted algebraic transformations.	MATH-500 algebra subset and GSM-Symbolic.
B. Restricted Python code	Sandboxed execution with fixed imports, fixed seeds, timeouts, and test inputs.	Bounded observational equivalence under a fixed sandbox and test distribution, not exact program equivalence.	HumanEval+ and MBPP+ as held-out evaluation.

Scope recommendation: Lead with algebra. Code is valuable, but code-state equivalence is inherently approximate. The cleanest first paper should prove the mechanism in algebra, then show code as a bounded-observational extension.

5. Experimental Design with Iso-Compute Accounting

All comparisons should report Pareto curves of accuracy versus total compute. Iso-compute must include token budget, model calls, wall-clock time, execution calls, and semantic-map overhead. Token budget alone is insufficient.

Method	Exec?	Online merge?	Value?	Purpose
Greedy	No	No	No	Minimal baseline.
Beam search	No	No	No	Surface-text beam baseline; vulnerable to semantic aliasing.
Best-of-N	No	No	No	Standard test-time sampling baseline.
Self-consistency	No	No	No	Majority voting over text reasoning paths.
Execution-guided decoding	Yes	No	No	Filters invalid transitions but does not merge redundant semantic states.
Verifier reranking / LEVER	Yes	No	Yes	Closest value-guided prior; scores or reranks after generation.
Tree of Thoughts	Optional	No	Optional	Text-level search with evaluation/backtracking; no state convergence.
Forward SAVI	Yes	Yes	No	Isolates state merging. This should be the main contribution.
Bidirectional SAVI	Yes	Yes	Yes	Tests whether P_bwd adds value beyond state merging.

Critical ablation: The core scientific split is Forward SAVI vs Bidirectional SAVI. Forward SAVI tests whether semantic-state merging alone helps. Bidirectional SAVI tests whether goal-conditioned value guidance adds incremental benefit.

5b. Metrics

Metric	Why it matters
Accuracy / pass@1	Primary task performance under matched compute budgets.
BAR_t — baseline beam aliasing rate	Primary diagnostic that standard text beams contain semantic duplicates.
CMR_t — SAVI candidate merge rate	Primary mechanism metric; shows how much SAVI merges before pruning.
RSD_t — retained semantic diversity	Fraction of beam capacity filled by unique states after pruning.
Invalid transition rate	Measures the value of execution pruning independently from merging.
False-merge and missed-merge rates	Audits the soundness and recall of the semantic map. Essential for code.
Tokens, model calls, execution calls	Compute accounting across generation and semantic-map operations.
Wall-clock time and memory	Prevents claiming efficiency while hiding execution overhead.
Markov diagnostic	Compare next-state distributions for merged histories using JS divergence, top-state overlap, or approximate KL.

5c. Detailed Experiment Settings

Parameter	Recommended setting	Rationale
Backbone	Qwen2.5-Math-7B-Instruct for algebra; Qwen2.5-Coder-7B-Instruct for code.	Open, domain-specific models support reproducibility.
Scale ablation	Repeat key runs with smaller and larger variants when feasible.	Tests whether merge rate and gains depend on model size.
Beam width K	4, 8, 16, 32.	Measures how aliasing scales with beam width.
Candidates per state N	4 per active state.	Enough diversity without runaway execution cost.

Max steps T	10 for algebra; 15 for code.	Keeps stepwise execution bounded.
Bidirectional lambda	0.3, 0.5, 1.0.	Sensitivity curve for P_bwd contribution.

5d. Semantic Map Settings

Domain A — Symbolic Algebra

- Use a restricted transformation grammar: one algebraic expression or equation transformation per step.
- Use SymPy canonicalization with declared assumptions. Restrict operations to `simplify`, `expand`, `factor`, `cancel`, `collect`, and substitution when valid.
- Set a strict timeout of 2 seconds per canonicalization call. Treat timeouts as invalid or unverified, not as equivalent.
- Audit 200 merged pairs manually or with an independent checker to estimate false-merge and missed-merge rates.

Domain B — Restricted Code Generation

- Use a Docker sandbox with no network access, no file I/O, limited memory, and a 2-second timeout per chunk.
- Whitelist only safe imports: `math`, `collections`, `itertools`, `functools`, `string`.
- Define state as bounded observational state: variable names/types/representations plus observed outputs on fixed public inputs.
- Do not claim exact program equivalence. Report equivalence as approximate under the sandbox and test distribution.
- For false-merge auditing, run both completed programs on additional held-out tests that were not used to define the merge state.

5e. Testable Hypotheses and Decision Rules

Hypothesis	Positive result	What it proves
H1: semantic aliasing exists	BAR_t > 10% for $K \geq 8$ on medium algebra problems.	The problem is real and worth solving. If BAR_t < 5%, narrow the scope or rethink the paper.
H2: Forward SAVI improves iso-compute efficiency	Forward SAVI at smaller K matches or exceeds beam search at larger K under equal total compute.	State merging increases useful semantic exploration.
H3: online merging beats post-hoc	Forward SAVI beats verifier reranking / post-hoc clustering under matched compute.	The trellis mechanism matters during generation, not just at final selection.

H4: P_bwd adds incremental value	Bidirectional SAVI beats Forward SAVI with statistical significance.	Goal-conditioned value guidance adds information beyond merging.
H5: Markov approximation is acceptable	Merged histories induce similar next-state distributions, or canonical prompting fixes the gap.	Viterbi-inspired trellis search is operationally sound.
H6: merge rate scales with beam width	BAR_t and CMR_t increase with K until a domain-specific ceiling.	Aliasing is structural rather than random.

Decision rule: H1 and H2 are required for a publishable Forward SAVI paper. H3 validates online merging. H4 is optional for the first paper. H5 protects the theory. H6 gives practical guidance on beam width.

6. Data Protocol and Leakage Controls

Test data — never train on these tasks or hidden tests

Dataset	Role
MATH-500	Held-out algebra evaluation. Use algebra-tagged subset first; expand later. Do not train P_bwd on these exact problems.
GSM-Symbolic	Held-out generalization test for symbolic arithmetic and template variation.
HumanEval+ / EvalPlus	Held-out code evaluation with dense tests. Hidden tests must not be used to train P_bwd.
MBPP+ / EvalPlus	Held-out code evaluation with broader problem styles. Keep evaluation tasks disjoint from P_bwd training tasks.

Training data — only for P_bwd, never for Forward SAVI

Dataset	Use	Leakage control
PRM800K / math trajectories	Train a state-value model from partial solution states and correctness labels.	Deduplicate against MATH-500/GSM-Symbolic by problem text and answer. Exclude overlaps.
NuminaMath or other math training corpora	Generate additional symbolic trajectories and labels.	Use only training problems; hold out benchmark tasks completely.
Magicode / Evol-Instruct / APPS-train	Collect partial execution states and final pass/fail labels.	Do not train on HumanEval+/MBPP+ evaluation task test outcomes. Use disjoint training tasks.

Leakage fix: Forward SAVI requires no training data. This is a major advantage. Bidirectional SAVI should be trained only on disjoint tasks and evaluated on held-out benchmarks. Never use evaluation-task hidden tests to train P_{bwd} .

7. P_{bwd} : Definition and Training Procedure

$P_{\text{bwd}}(G=1 \mid S_t, \text{problem})$ is a goal-conditioned state-value model trained as a binary classifier over intermediate states. It estimates whether a current state is on a path likely to solve the task.

- **Input:** canonical state S_t plus the original problem statement.
- **Output:** calibrated probability that the trajectory can reach a correct final answer.
- **Training label for math:** final-answer correctness and/or step-level process labels on disjoint training tasks.
- **Training label for code:** final program pass/fail on training-task tests, not benchmark hidden tests.
- **Model size:** start with 1B parameters or smaller; scale only if validation performance is poor.

Strategic framing: Do not let P_{bwd} dominate the paper. The main novelty is state merging. P_{bwd} is useful only if H4 shows clear incremental gain beyond Forward SAVI.

8. Novelty Against Prior Work

Prior work	What it does	SAVI distinction
Execution-guided decoding	Uses execution to detect and exclude invalid partial programs during decoding.	SAVI uses execution to define the search object itself. EGD prunes; SAVI merges duplicate semantic states and redirects budget.
LEVER / verifier reranking	Generates programs, executes them, and scores or reranks outputs.	SAVI merges intermediate equivalent states online, changing what gets generated next.
Tree of Thoughts	Searches over coherent text-level thoughts with evaluation and backtracking.	SAVI searches over canonical executable states. The search geometry is a trellis, not a text tree.
Process reward models	Score intermediate reasoning steps.	SAVI can use PRMs as value models, but the distinctive mechanism is state merging.
AlphaCode clustering	Samples many programs and clusters by final execution behavior.	SAVI performs online intermediate-state merging, not final-step clustering only.
MCTS for code/math	Builds a search tree with learned value functions.	SAVI nodes are canonical executable states; equivalent text nodes are merged rather than duplicated.

9. Strongest Paper Framing

One-sentence contribution: SAVI converts test-time scaling from generating more textual variants into exploring more distinct executable semantic states, changing the search geometry from a tree of text histories to a trellis of semantic states.

The clean first paper should prove the claims in this order:

- **Semantic aliasing exists and is measurable:** report BAR_t across beam widths, problem difficulty, and step index.
- **State merging improves iso-compute efficiency:** Forward SAVI reaches equal or better accuracy with fewer wasted slots.
- **Online merging beats post-hoc reranking or final clustering:** this validates the trellis mechanism.
- **Goal-conditioned guidance is optional:** include Bidirectional SAVI only if it adds statistically significant gain over Forward SAVI.

Recommended title	Comment
From Trees to Trellises: State-Aware Viterbi Inference for Executable Reasoning	Best title. Memorable and accurate.
SAVI: Scaling Test-Time Compute via Semantic-State Search	Good short title.
Semantic-State Beam Decoding: From Textual Diversity to Reasoning Diversity	Most literal title.

10. Immediate Implementation Plan

Phase	Goal	Deliverable
Phase 1A	Implement algebra Phi with restricted SymPy transformations.	Working Forward SAVI prototype with BAR_t , CMR_t , RSD_t , invalid-rate logs.
Phase 1B	Run MATH-500 algebra subset at $K = 4 / 8 / 16 / 32$.	Merge-rate curves and iso-compute accuracy curves.
Phase 1C	Compare online merging to post-hoc merging / reranking.	Evidence for or against H3.
Phase 2	Extend to bounded observational code equivalence.	Sandboxed code-state results with false-merge audit.
Phase 3	Train P_{bwd} on disjoint training tasks.	Bidirectional SAVI ablation only after Forward SAVI is validated.

Priority: The next move is empirical, not rhetorical: run Phase 1A/1B and measure BAR_t and CMR_t. If merge rates exceed roughly 20-30% at useful beam widths, SAVI becomes a serious test-time compute paper.

11. References for Positioning

- [1] Wang et al. *Robust Text-to-SQL Generation with Execution-Guided Decoding*. arXiv:1807.03100, 2018.
- [2] Ni et al. *LEVER: Learning to Verify Language-to-Code Generation with Execution*. arXiv:2302.08468 / ICML 2023.
- [3] Yao et al. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. arXiv:2305.10601 / NeurIPS 2023.
- [4] Lightman et al. *Let's Verify Step by Step*. arXiv:2305.20050, 2023. PRM800K.
- [5] Li et al. *Competition-Level Code Generation with AlphaCode*. Science 2022.
- [6] Wang et al. *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. arXiv:2203.11171 / ICLR 2023.
- [7] Chen et al. *Evaluating Large Language Models Trained on Code*. arXiv:2107.03374, 2021. HumanEval.
- [8] Hendrycks et al. *Measuring Mathematical Problem Solving with the MATH Dataset*. arXiv:2103.03874, 2021.
- [9] Liu et al. *CodeAct: Enabling LLM Agents that Act via Code Execution*. arXiv:2402.01030, 2024.
- [10] Liao et al. *EvalPlus: Rigorous Evaluation of Code LLMs*. arXiv:2305.01210, 2023.
- [11] Wei et al. *Magicoder: Source Code Is All You Need*. arXiv:2312.02120, 2023.

SAVI Fixed Research Positioning Memo v6 | Uniform 10pt Helvetica black. Key v6 fixes: corrected complexity lemma, three-metric split (BAR/CMR/RSD), P_bwd DPI claim removed, code equivalence bounded, leakage controls added.